Self-evolving agents with frozen models

"调研：冻结模型下Agent持续自进化，失败轨迹→作用域知识→使用时注入→配对门禁，但来源题不翻转、接受0、不遵循、组件无效、与隐藏断言冲突。找前人方案：1失败→可执行技能/工具包装；2信用分配与根因；3自动合成契约/断言/测试；4工具调用合规强制；5影响面回归门禁与小样本高功效；6记忆退化与预算对齐。关键词：self-evolving agents, harness evolution, skill acquisition, tool-use compliance, verifier synthesis, regression test selection, ACE, SAGE, StarHarness。输出每类最强方案、适用条件、差异、可迁移做法、失败教训。" *Generated by LeapSpace*

冻结模型下Agent持续自进化：关键结论速览

Key Findings Table

主题	最强方案/组合	适用条件	主要优势	主要短板/失败教训
1. 失败→可执行技能/工具包装	AWM；补充 SAMULE / FORGE / MACLA	任务结构可抽象、失败模式重复、后续任务可检索复用 [1] [2] [3]	AWM把经验显式化为可复用workflow，在Mind2Web提升24.6%、WebArena提升51.1%，且跨任务/跨域泛化更强 [1]	仅重试通常脆弱；记忆会因泛化过强、冗余、冲突、过时而污染 [4] [5] [6]
2. 信用分配与根因定位	StepFinder + Scope Delineation Before Localization；基准为 Who&When	有完整轨迹、可按step/stage切分、可标注责任agent [7] [8] [9]	先定scope再定decisive step，可把step-level accuracy进一步提高，StepFinder同时显著降推理成本 [7] [9]	精确step定位仍难；多因一果、外生故障、评测噪声会显著拉低效果 [8] [10] [11]
3. 自动合成契约/断言/测试	Precis / DbC-GPT / PBACE / PRECA 组合	有可执行verifier、形式模型、或可重放环境 [12] [13] [14] [15]	反例驱动可把隐式要求转成可执行gate；Precis给generator-relative correctness，DbC-GPT能用验证器反例迭代收紧postconditions [12] [13] [16]	动态推断受测试质量限制；符号执行/Concolic在复杂状态、循环、数组上易爆炸 [17] [18] [19]
4. 工具调用合规强制	协议网关 + host-side access control + intent-bound guard + sandbox	工具调用可被中介、参数结构化、执行面可封装 [20] [21] [22] [23]	运行时硬治理优于prompt约束；可阻断schema/policy/security/越权类违规 [20] [24] [25]	schema/授权只能挡格式与越权，挡不住授权范围内的恶意语义 [26] [27]
5. 影响面回归门禁与小样本高功效	impact-aware RTS + budget-aware prioritization；预算紧时用 DeepARTS / XGRL-TCP / COLEMAN / LARA-TCP	可获得trace、覆盖、依赖或契约变更信息 [28] [29] [30] [31]	小集合高功效来自“受影响面选择”而非随机抽样；能更早抓错并减少无效重跑 [30] [31] [32] [33]	覆盖不足时，impact analysis会漏测 [34]
6. 记忆退化与预算对齐	MemCurrency + EvoMem + dynamic topic-aware retrieval	长程任务、多轮积累、上下文预算受限 [35] [36] [37]	freshness/相关性/使用频率联合加权，优于静态分桶和简单压缩；固定预算下仍能抑制stale recall [35] [36] [37]	记忆越多不一定越好；session recall高不等于fact recall高 [38]

Direct Answer

你的问题，现有证据支持的**最可落地前人方案**不是“让冻结模型自己想通”，而是把失败外化成**可复用artifact**，并在注入前后都加**验证与门禁**。综合来看，最佳实践链路是：

失败轨迹 → failure scope划定 → decisive-step/责任组件定位 → 产出artifact（workflow / reflection / manual / contract / test）→ freshness/兼容性过滤 → 使用时检索注入 → 配对回归门禁 [1] [7] [9] [13] [24] [29] [33] [39]。

按你要的六类“最强方案”：

1. **失败→技能/工具包装**：最强证据是 AWM，因为它把经验转成可复用workflow，并且有明确跨任务收益；若你更偏“失败归纳反思知识”，则 SAMULE 更合适；若你更偏长期记忆库，则 FORGE/MACLA 更适合 [1] [2] [3] [39]。
2. **信用分配与根因**：最强组合是 Who&When + StepFinder + Scope-first localization，因为它们直接面向“谁错、错在哪一步”，而非只看最终是否失败 [7] [8] [9]。
3. **契约/断言/测试自动合成**：最强不是单一方法，而是 Precis/PRECA/DbC-GPT/PBACE 这类“契约推断 + 反例精化 + 可执行测试”的组合 [12] [13] [14] [15]。
4. **工具调用合规强制**：最强是**运行时分层治理**，即 protocol gateway、host-side策略层、intent-bound guard、sandbox，不靠prompt软约束 [20] [21] [22] [23]。
5. **影响面回归门禁与小样本高功效**：最强是affected-test selection + budget-aware prioritization；如果你有契约/依赖/trace，就先做影响分析；若预算极紧，再上DeepARTS、XGRL-TCP、COLEMAN等排序器 [30] [31] [32] [33] [40]。
6. **记忆退化与预算对齐**：最强是 MemCurrency + EvoMem + topic-aware retrieval，核心不是“存更多”，而是“只注入新鲜、相关、预算内且不冲突的记忆” [35] [36] [37]。

对你列出的具体失败型态——“来源题不翻转、接受0、不遵循、组件无效、与隐藏断言冲突”——语料没有以这些原词建模，但可以分别映射到**指令不遵循、无效工具执行、契约/隐藏断言冲突、回归泄漏、授权范围内语义违规**等邻近问题 [26] [29] [33]。
另外，ACE、SAGE、StarHarness 这几个命名系统，在提供证据里没有可核验的对应论文，不能据此做实体化比较 [meta_analisys.unable_to_answer 不可直接引用，故不作论文归属断言]。

结论性建议：如果你要做冻结模型下的持续自进化，最可迁移的做法是：**只把“已定位、已验证、可执行”的修复写回长期记忆；注入时必须过freshness/compatibility gate；执行前后必须各有一层硬门禁** [9] [16] [24] [35] [41]。
Confidence level: High - 六条主线都有较一致证据支撑，但个别“最强方案”的实证粒度不均衡 [1] [7] [12] [20] [30] [35]。

Study Scope

- 时间范围：主要覆盖 2019-2025，少量基础工作更早，用于方法谱系说明 [42] [43] [44]。
- 学科范围：LLM agents、多智能体、软件验证、契约推断、测试生成、运行时治理、回归测试选择、长期记忆管理 [12] [20] [45] [46]。
- 方法类型：benchmark对比、系统机制论文、模型驱动测试、运行时 enforcement、记忆/检索框架、根因定位数据集与算法 [8] [14] [24] [35]。

Assumptions & Limitations

- 现有证据更强于“组件级机制”，弱于“六类能力在同一agent harness中端到端闭环”的统一验证 [29] [47]。
- 若没有高质量轨迹、变更可观察性、或可执行verifier，很多方案难以稳定迁移 [17] [34] [48]。
- 工具合规门禁与记忆门禁常带来效用损失、延迟增加或路由脆弱性 [27] [49] [50]。

Suggested Further Research

- 把 **failure attribution** 直接接到 **memory write-back policy**，只提升“已定位且已验证”的修复 [6] [9]
- 为 agent harness 建立 **contract-sensitive change model**，覆盖prompt schema、tool policy、memory entries、orchestrator logic [29] [46]
- 度量工具合规门禁的 **false-positive / false-negative / utility-loss** 曲线，而非只报阻断成功 [49] [51]
- 将隐藏断言显式化为 **pre/postconditions + counterexample loop**，并与回归选择联动 [13] [16] [33]
- 研究“来源题不翻转/接受0/组件无效”等 agent-specific failure semantics 的标准数据集与标注协议 [8] [52]

完整报告

1. 引言：问题定义与综述边界

冻结模型下的Agent持续自进化，本质上是在**不改权重**条件下，把失败转成可复用外部能力，并防止这些新增能力破坏原有行为或越过安全边界 [20] [45] [53]。现有证据表明，最稳的路线不是单轮自反思，而是把改进写成**workflow、reflection、manual、example、contract、test** 等外部artifact，再通过检索与运行时治理注入 [1] [12] [24] [39] [54]。

你的问题还强调“来源题不翻转、接受0、不遵循、组件无效、与隐藏断言冲突”。从已有文献可映射为：**指令/策略不遵循、错误接受低价值输出、组件/tool接口失配、运行时无效动作、隐式契约未被显式验证** [26] [33] [55]。因此，综述重点应从“自我改进”扩展到“**改进后的合规与回归门禁**”。

Confidence level: High - 该问题定义与现有研究方向高度一致，且跨主题可互证 [1] [8] [22] [35]。

2. 基础语境：六类机制如何拼成一个冻结模型闭环

可把冻结模型下的持续自进化分成六个机制层：

1. **失败外化层**：把失败轨迹转换为workflow、manual、reflection、example或程序化恢复策略 [1] [47] [54]。
2. **归因层**：确定是哪个agent、哪个stage、哪个step导致失败 [8] [9] [56]。
3. **验证层**：把隐式要求变成契约、断言、测试，用反例或模型检查收紧 [12] [13] [16]。
4. **执行治理层**：在真正调用工具前后做schema、policy、intent、sandbox检查 [22] [23] [24]。

5. **回归门禁层**：只跑受影响而高功效的小集合，阻止局部修复引入全局退化 [28] [30] [33]。
6. **记忆预算层**：决定什么值得存、何时注入、如何在预算内防止stale/conflict [35] [36] [37]。

Taken together, these findings suggest：**真正有效的"自进化"其实是"外部artifact演化 + 运行时验证治理"**，而不是模型内部能力自动生长 [24] [29] [57]。
Confidence level: High - 该结构来自多个主题的交叉收敛，而非单篇论文结论 [1] [9] [13] [20] [33]。

3. 失败→可执行技能/工具包装

3.1 最强方案

AWM 是当前最强的"失败/经验→可执行workflow"证据，因为它不只是存记忆，而是归纳出可复用流程，并有清晰的跨任务增益：Mind2Web相对成功率提升24.6%，WebArena提升51.1%，且分布差距越大时泛化收益越明显 [1]。

SAMULE 的强点不在workflow，而在**三层反思粒度**：单轨迹修正、同任务错误分类、跨任务同类错误迁移，适合把失败归纳成"作用域知识"而非固定流程 [39]。

FORGE / MACLA / U-EHR 更像长期知识库路线：把失败转成规则、例子、混合记忆或procedure，并用群体传播、Bayesian可靠度估计、UCB检索选择提高复用稳定性 [2] [3] [58]。

3.2 适用条件与差异

- **workflow级**：适合网页、复杂操作、步骤可重放任务 [1]
- **reflection级**：适合错误模式多、任务表面差异大但根因相近的场景 [39]
- **manual级**：适合环境知识稀缺、但可由少量交互抽取规则的场景 [54]
- **example/rule库**：适合需要快速ICL注入、低工程成本部署 [2] [59]

3.3 可迁移做法与失败教训

最可迁移的做法是：把一次失败写成**失败签名 + 适用前提 + 修复步骤 + 反例条件 + 可验证结果**，而不是只存自然语言总结 [60] [61]。
失败教训则很一致：**retry-only** 脆弱；没有grounding/verification的反思容易徒增开销；泛化过强或碎片化检索会污染后续决策 [4] [5] [6] [62] [63]。

结论：若你的目标是"失败轨迹→作用域知识→使用时注入"，首选workflow/structured memory，而不是无结构反思文本 [1]。
Confidence level: High - AWM等方法有明确实证，且失败模式在多篇中重复出现 [1] [4] [6]。

4. 信用分配与根因定位

Who&When 建立了"失败责任agent + decisive error step"的基准，是此方向的核心数据基础 [8]。其结果也说明难点：最佳方法的agent识别准确率只有53.5%，step预测只有14.2%，说明"知道失败了"和"知道哪一步错了"是两回事 [8]。

在方法上，**StepFinder** 把LLM主要用于特征构建，再做时序建模与注意力打分，兼顾效果与效率，较既有LLM-based方法显著降推理时间 [7]。
Scope Delineation Before Localization 则指出：直接找错步常常不准，应该先缩小failure scope，再在scope内做step定位，可把step-level accuracy再提高最多24.27% [9]。

若你做生产链路诊断，**Planner-Executor-Critic** 一类traceability框架更适合，因为它关注错误在哪个stage启动、如何传播，并报告repair rate与harm rate [56]。
若你更关心“哪些失败值得写回记忆”，**AgentDiagnose** 有价值，因为它把backtracking、task decomposition、observation reading、self-verification等能力维度与人工判断相关联 [64]。

失败教训主要有三类：

- **多因一果**：很难只归因给单step或单agent [65] [66]
- **外生故障**：部分失败不在agent本身，17.8%可仅由环境/参考/judge问题解释 [10]
- **自动判责偏差**：单轮判断易确认偏误，且与人工判断可能弱相关 [11] [67]

结论：你的系统若要“失败后自动升格成知识”，必须先做**scope-first credit assignment**，否则写回记忆会放大污染 [6] [9]。
Confidence level: High - 数据集、评价协议、局限都较清晰 [8] [10] [64]。

5. 自动合成契约、断言与测试

这条线对“隐藏断言冲突”最关键。文献显示，最可靠的做法不是让LLM口头判断“修好了”，而是把隐式要求转成**可执行契约和测试** [13] [16]。

5.1 最强方案

- **PRECA**：从输入输出观测推断preconditions，并明确声称有正确性保证 [15]。
- **Precis**：学习“tight contracts”，但正确性是相对于test generator的 [12]。
- **DbC-GPT**：从自然语言抽取Solidity postconditions，再用验证器反例迭代收紧 [13]。
- **PBACE**：把JSON描述EFSM转成可执行测试序列，适合状态机/流程型agent harness [14]。

5.2 适用条件与差异

- 有**形式模型/状态机**时，EFSM/PBACE最强 [14]
- 有**verifier**时，DbC-GPT/contract strengthening/Metamorph更强 [13] [68] [69]
- 只有**动态轨迹/测试**时，Precis、EvoSpex等更现实，但受测试质量制约 [12] [70]

5.3 失败教训

动态推断的核心弱点是：**如果测试不够强，错误契约也可能看似成立** [16] [17]。因此需要rich counterexamples，而非只有yes/no的失败信号；最佳反例是能返回变量轨迹、状态机或行为拓扑信息的 [16]。
另一方面，符号执行与concolic虽然强，但在循环、复杂结构、数组、过程调用上可扩展性很差 [17] [18] [19] [71]。

结论：若你担心“来源题没翻转却被接受0”或“隐藏断言冲突”，应把接受标准改写为**pre/postconditions + counterexample-guided refinement**，而不是让agent自己判通过 [13] [16]。
Confidence level: High - 软件验证侧证据成熟，但迁移到agent harness仍需工程化适配 [12] [29]。

6. 工具调用合规强制

该主题证据最一致：**prompt约束不够，必须有运行时硬治理** [20] [24]。

6.1 最强机制谱系

1. **Protocol gateway**：对每次工具调用做schema、语义兼容、policy、审计校验 [20] [24] [26]
2. **Host-side access control**：在不改agent和服务端的前提下，做scoped access、只读默认、审批提升 [21]
3. **Intent-bound guard**：依据意图偏移、参数风险、链式异常做allow/downgrade/human-confirm/deny [22]
4. **Floor-guardrail**：保证执行动作不能低于策略规定的最小升级/审批级别 [72]
5. **Sandbox**：高风险动作先在隔离环境观察文件、网络、进程、权限副作用 [23] [73]

6.2 适用条件与差异

若你的问题是“不遵循、组件无效、参数篡改”，最直接可迁移的是**把prompt规则改成执行前硬检查**：

- 参数合法性：schema + safe query lifecycle [74]
- 越权：identity/token/policy flow [25]
- 语义偏航：intent-bound/risk scoring [22] [75]
- 高风险副作用：sandbox + HITL [23] [76]

6.3 已知失效边界

最重要的负面结论是：**schema valid ≠ semantic safe**。授权身份、合法参数、合规格式，仍可能执行“授权范围内的恶意意图” [26]。因此需要typed separation、sink-aware policy、tool output与instruction分离、后果导向策略 [26] [77]。

结论：对“接受0、不遵循、组件无效”类问题，最有用的不是更强prompt，而是**边界治理层** [20] [22]。
Confidence level: High - 多种架构都指向同一结论，但精细FP/FN曲线证据仍不足 [51] [78]。

7. 影响面回归门禁与小样本高功效

文献一致反对随机抽样式回归。最有效的是：**先找受影响面，再在预算内排序** [30] [32]。

7.1 最强方案

- **影响分析/RTS**：基于signature、contract、implementation、trace links选择affected tests，可显著减少重跑且保留fault-detection能力 [28] [32] [33] [79]
- **结构化agent harness testing**：用trace、mocking和assertions对agent组件与交互做回归验证，特别适合orchestrator/prompt/tool层变更 [29]
- **预算排序器**：DeepARTS、XGRL-TCP、COLEMAN、LARA-TCP在低预算下更早抓错 [30] [31] [40] [80]

7.2 可迁移做法

针对“来源题不翻转/隐藏断言冲突”，最好的迁移做法是把它们编码成：

- contract-sensitive affected tests [33] [46]
- trace assertions on orchestrator interactions [29]
- 必要时加入t-way**组合测试**捕捉隐藏交互缺陷 [81]

7.3 失败教训

最大问题不是排序器不够聪明，而是**覆盖与可观察性不够**。如果依赖覆盖太浅，impact analysis会漏掉真实受影响测试 [34]。

结论：你的“配对门禁”应设计为：**注入前合法性门禁 + 注入后受影响面回归门禁**，而不是全量回归或随机抽测 [24] [33]。
Confidence level: High - 经典CI证据较强，向agent harness迁移的结构化测试证据也已出现 [29] [32]。

8. 记忆退化与预算对齐

长期自进化系统的瓶颈并非“不会记”，而是“**记错、记旧、记太多、记冲突**” [5] [6] [35]。

8.1 最强方案

- MemCurrency：按使用频率、语义相关性、新鲜度给记忆打currency weight，并在冲突出现时刷新 [35]
- Dynamic topic-aware retrieval：优于无结构检索、静态topic分桶、简单context compression [37]
- EvoMem：在固定预算下选择salient timepoints并摘要，抑制过时答案 [36]

8.2 预算与执行对齐

证据显示，预算管理有效手段包括：

- history trimming / context compression [82] [83]
- adaptive max_tokens [84]
- structured data优于冗长自然语言 [84] [85]
- 顺序式/动态式流程通常比并行prompt更省token且质量更稳 [50] [86]

8.3 失败教训

- 过时、冗余、冲突记忆会污染后续决策 [5] [6]
- session recall高不等于事实回忆强 [38]
- topic routing仍可能出错 [37]
- 仅靠重放/隐藏状态重建在漂移下会失败，需要短期FIFO、确定性恢复路径、非推测模式等回退 [87] [88] [89]

结论：注入前必须做**freshness gate + conflict gate + budget gate**；没有这三层，持续写回会把系统越修越脏 [6] [35] [36]。
Confidence level: High - 多篇论文对stale/conflict/budget问题结论一致 [35] [36] [37]。

9. 综合比较：六类问题的最佳组合与设计原则

9.1 每类最强方案与适用条件

- **失败→技能**：AWM；适合可流程化任务 [1]
- **失败→反思知识**：SAMULE；适合错误模式迁移 [39]
- **判责定位**：Scope-first + StepFinder；适合长轨迹多步任务 [7] [9]
- **契约/测试**：DbC-GPT + Precis/PRECA + PBACE；适合有verifier或模型的场景 [12] [13] [14] [15]
- **工具合规**：协议网关 + host-side policy + sandbox；适合多工具高风险环境 [20] [21] [23]
- **回归门禁**：affected-test selection + budget prioritization；适合经常改harness的CI/CD [30] [33]
- **记忆预算**：MemCurrency + EvoMem + topic-aware retrieval；适合长生命周期agent [35] [36] [37]

9.2 可迁移设计原则

1. **结构化失败日志**：至少记录step、tool、参数、证据、结果 [8] [61]
2. **先定位再写回**：只把已局部化失败转成长期知识 [9] [64]
3. **知识双轨制**：同时保存人可读memory与机可检contract/test bundle [2] [16]
4. **分层门禁**：注入前做freshness/compatibility；执行前做policy/schema/intent；执行后做affected regression [24] [33] [35]
5. **预算感知选择**：优先保留高功效测试与高价值记忆，而非全量保留 [31] [36]

9.3 失败教训

- 反思若无外部验证，常变成昂贵噪声 [24] [62]

- 安全门禁会带来效用损失与延迟，需显式权衡 [27] [49]
- 回归门禁若缺乏可观察性，会漏掉真实冲突 [34]
- 记忆系统若缺freshness/conflict治理，会自我污染 [6] [35]

总判断：最强前人方案不是单篇论文，而是一个**组合栈**：
AWM/SAMULE式artifact化 + Who&When/StepFinder式归因 + DbC-GPT/Precis/PBACE式验证合成 + 协议网关/intent guard/sandbox式合规强制 + impact-aware RTS式回归门禁 + MemCurrency/EvoMem式记忆预算治理 [1] [7] [9] [12] [13] [20] [22] [33] [35] [39]。
Confidence level: High - 组合结论为跨文献综合推断，但各部件的局部证据较强且相互兼容 [1] [8] [16] [26] [32]。

10. 结论

冻结模型下，Agent持续自进化最可靠的路线是：**把失败变成外部、可验证、可检索、可回归的artifact**，再用运行时门禁和预算治理保证这些artifact不会把系统带偏 [1] [16] [20] [35]。
如果只做记忆注入，不做归因过滤、契约显式化、工具硬治理和受影响面回归，那么你担心的“不翻转却通过、接受0、不遵循、组件无效、隐藏断言冲突”都仍会反复出现 [11] [26] [34]。

最终结论的信心：High - 主要因为六个子问题虽来自不同社区，但收敛到同一工程原则：**外化、验证、分层治理、预算感知** [12] [22] [30] [36] [41]。

References

1. Wang, Z.Z., Mao, J., Fried, D., Neubig, G. (2025) Agent Workflow Memory. *Proceedings of Machine Learning Research*. https://www.scopus.com/pages/publications/105023505273
2. Bogdanov, I., Lung, C.-H., Kunz, T., (...), Zaman, M. (2026) FORGE: Self-Evolving Agent Memory with No Weight Updates via Population Broadcast. *Proceedings of the ACM Conference on AI and Agentic Systems, CAIS 2026*. https://doi.org/10.1145/3786335.3813155
3. Forouzandeh, S., Peng, W., Moradi, P., (...), Jalili, M. (2026) Learning Hierarchical Procedural Memory for LLM Agents through Bayesian Selection and Contrastive Refinement. *AAMAS 2026 - Proceedings of the 25th International Conference on Autonomous Agents and Multiagent Systems*. https://doi.org/10.65109/FKYO8341
4. Bâra, A., Dobrița, G., Oprea, S.-V. (2026) Think it, Run it: Autonomous machine learning pipeline generation via self-healing multi-agent artificial intelligence. *Knowledge-Based Systems*. https://doi.org/10.1016/j.knosys.2026.116688
5. Teng, L., Ni, W., Song, L., (...), Li, Y. (2026) Towards Self-Evolving Agents: A Dual-Process Framework for Continual Context Refinement. *Electronics (Switzerland)*. https://doi.org/10.3390/electronics15061232

6. Liu, Q., Wang, M., Xie, J. (2026) Failure-Gated Hierarchical Memory: Preventing Memory Pollution in Long-Horizon LLM Agents via Critic-Guided Write Control. *2026 6th International Conference on Machine Learning and Intelligent Systems Engineering, MLISE 2026*. https://doi.org/10.1109/MLISE70044.2026.11607496
7. Zhu, T., Wu, Y., Jin, W., (...), Huang, G. (2026) StepFinder: A Temporal Semantic Framework for Failure Attribution in Multi-Agent Systems. *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. https://doi.org/10.1145/3770855.3817991
8. Zhang, S., Yin, M., Zhang, J., (...), Wu, Q. (2025) Which Agent Causes Task Failures and When? On Automated Failure Attribution of LLM Multi-Agent Systems. *Proceedings of Machine Learning Research*. https://www.scopus.com/pages/publications/105023465594
9. Sun, K., Li, W., Dong, B., (...), Shi, B. (2026) Scope Delineation Before Localization: A Two-Stage Framework for Enhancing Failure Attribution in Multi-Agent Systems. *Proceedings of the AAAI Conference on Artificial Intelligence*. https://doi.org/10.1609/aaai.v40i39.40594
10. Hellin, S., Jang, S., Fuchs, S., (...), Borrmann, A. (2026) Agentic search for BIM information extraction: Systematic evaluation. *Automation in Construction*. https://doi.org/10.1016/j.autcon.2026.107260
11. Chen, Y., Liu, B., Li, Y., (...), Huang, L. (2027) A trustworthy agentic AI framework for auditable label-free machinery fault diagnosis in safety-critical engineering systems. *Advanced Engineering Informatics*. https://doi.org/10.1016/j.aei.2026.105205
12. Astorga, A., Saha, S., Dinkins, A., (...), Xie, T. (2021) Synthesizing contracts correct modulo a test generator. *Proceedings of the ACM on Programming Languages*. https://doi.org/10.1145/3485481
13. Leite, G., Arruda, F., Antonino, P., (...), Roscoe, A.W. (2024) Extracting Formal Smart-Contract Specifications from Natural Language with LLMs. *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. https://doi.org/10.1007/978-3-031-71261-6_7
14. Pan, Z., Shu, T., Ding, Z. (2023) A Parsing-Based Approach for Constructing Executable EFSM. *Proceedings - 2023 9th International Symposium on System Security, Safety, and Reliability, ISSSR 2023*. https://doi.org/10.1109/ISSSR58837.2023.00058
15. Menguy, G., Bardin, S., Lazaar, N., Gotlieb, A. (2023) Automatic Code Analysis: Revisiting precondition inference via constraint acquisition. *18es Journees Francophones de Programmation par Contraintes, JFPC 2023 - Held at Plate-Forme Intelligence Articielle, PFIA 2023*. https://www.scopus.com/pages/publications/85207865543
16. Iannopollo, A., Incer, I., Sangiovanni-Vincentelli, A.L. (2024) Synthesizing LTL contracts from component libraries using rich counterexamples. *Science of Computer Programming*. https://doi.org/10.1016/j.scico.2024.103116
17. Lonetti, F., Marchetti, E. (2018) Emerging Software Testing Technologies. *Advances in Computers*. https://doi.org/10.1016/bs.adcom.2017.11.003

18. Xiao, Xusheng (2012) Advances on Improving Automation in Developer Testing. *Advances in Computers*. https://doi.org/10.1016/B978-0-12-396526-4.00004-7

19. Zhang, D., Liu, D., Lei, Y., (...), Wang, W. (2012) SimFuzz: Test case similarity directed deep fuzzing. *Journal of Systems and Software*. https://doi.org/10.1016/j.jss.2011.07.028

20. Wang, S., Zhu, S., Li, R. (2026) Runtime Policy Enforcement for MCP-Based LLM Agents. *Electronics (Switzerland)*. https://doi.org/10.3390/electronics15132829

21. Singh, G., Madisetti, V.K. (2026) MCP-Secure: A Runtime Access Control Layer for Privilege-Aware LLM Agent Tooling. *IEEE Open Journal of the Computer Society*. https://doi.org/10.1109/OJCS.2026.3664314

22. Li, X., Zhao, X., Weng, Z., (...), He, S. (2026) IBBC-Guard: Intent-Bound Behavior-Chain Modeling for Unauthorized Tool-Call Detection and Dynamic Permission Control in LLM Agents. *2026 3rd World Conference on Computer and Information Security, WCCIS 2026*. https://doi.org/10.1109/WCCIS70285.2026.11650697

23. Jiang, J. (2026) Runtime Protection Practice for LLM Agent Security: An On-Demand Behavioral Sandbox Approach. *2026 11th International Symposium on Advances in Electrical, Electronics and Computer Engineering, ISAEECE 2026*. https://doi.org/10.1109/ISAEECE70502.2026.11635191

24. Kirubakaran, A.M., Punniyamoorthy, V., Deshpande, A., (...), Carimireddy, S. (2026) Governed Agentic Cloud Data Pipelines with MCP Gateways. *2026 International Conference on Artificial Intelligence, Systems, and Emerging Technologies, ICAISET 2026*. https://doi.org/10.1109/ICAISET66439.2026.11541321

25. Bhushan, B., Pappu, K., Jadav, V., Jaiswal, N. (2026) Risk-Adaptive Authorization for Agentic AI Systems. *Conference Proceedings - IEEE SOUTHEASTCON*. https://doi.org/10.1109/SoutheastCon63549.2026.11476129

26. Stodt, F., Reich, C., Stodt, J. (2026) A Federated MCP Gateway: Enforceable Isolation and Measuring Residual Semantic Attacks. *Proceedings of IEEE/IFIP Network Operations and Management Symposium, NOMS 2026*. https://doi.org/10.1109/NOMS69089.2026.11668221

27. Huang, S., Lee, S.-J. (2026) Bringing the Dual-LLM pattern to practice in Google ADK for deterministic AI agent security against prompt injection. *Artificial Life and Robotics*. https://doi.org/10.1007/s10015-026-01152-3

28. Khan, T.A., Heckel, R. (2011) On model-based regression testing of web-services using dependency analysis of visual contracts. *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. https://doi.org/10.1007/978-3-642-19811-3_24

29. Kohl, J., Kruse, O., Mostafa, Y., (...), Laurent-Winter, C. (2025) Automated Structural Testing of LLM-Based Agents: Methods, Framework, and Case Studies. *Proceedings of the IEEE International Conference on Big Data, BigData*. https://doi.org/10.1109/BigData66926.2025.11401679

30. Kongarana, S.R., Ananda Rao, A.R., Palagiri, R.R. (2026) DeepARTS: A Graph-Transformer Continual Learning Framework for Adaptive Regression Test Sequence Recommendation. *Journal of Advances in Information Technology*. https://doi.org/10.12720/jait.17.3.534-542

31. Lima, J.A.P., Vergilio, S.R. (2020) Multi-Armed Bandit Test Case Prioritization in Continuous Integration Environments: A Trade-off Analysis. *ACM International Conference Proceeding Series*. https://doi.org/10.1145/3425174.3425210

32. Mansour, N., Bahsoon, R. (2002) Reduction-based methods and metrics for selective regression testing. *Information and Software Technology*. https://doi.org/10.1016/S0950-5849(02)00027-7

33. Lou, Y., Chen, J., Zhang, L., Hao, D. (2019) A Survey on Regression Test-Case Prioritization. *Advances in Computers*. https://doi.org/10.1016/bs.adcom.2018.10.001

34. Hejderup, J., Gousios, G. (2022) Can we trust tests to automate dependency updates? A case study of Java Projects. *Journal of Systems and Software*. https://doi.org/10.1016/j.jss.2021.111097

35. Su, B., Wang, S., Pi, S., (...), Jiang, Z. (2026) MemCurrency: Memory Decay and Refresh Mechanism for Long-Running LLM Agents. *2026 11th International Conference on Electronic Technology and Information Science, ICETIS 2026*. https://doi.org/10.1109/ICETIS70504.2026.11633479

36. Wang, Z., Wang, C., Luo, H., (...), Wang, B. (2026) EvoMem: Timeline-Grounded Validity Adjudication for Non-Monotonic Multi-Agent Memory. *SIGIR 2026 - Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval*. https://doi.org/10.1145/3805712.3809931

37. Trofimov, A., Butakov, N., Maksimov, A., Maria, K. (2027) Memory Mining for LLM Agents: Dynamic Topic-Aware Retrieval of User Fact Records. *Lecture Notes in Computer Science*. https://doi.org/10.1007/978-3-032-29292-6_19

38. Sidik, B., Rokach, L. (2026) MEMTIER: Tiered Retrieval, Session-Level Injection, and Typed Consolidation for Long-Running LLM Agents. *Algorithms*. https://doi.org/10.3390/a19070607

39. Ge, Y., Romeo, S., Cai, J., (...), Zhang, Y. (2025) SAMULE: Self-Learning Agents Enhanced by Multi-level Reflection. *EMNLP 2025 - 2025 Conference on Empirical Methods in Natural Language Processing, Proceedings of the Conference*. https://doi.org/10.18653/v1/2025.emnlp-main.839

40. Kongarana, S.R., Ananda Rao, A.R., Raju, R.P. (2025) XGRL-TCP: An Explainable Graph-Based Reinforcement Learning Framework for Test Case Prioritization in CI. *Mathematical Modelling of Engineering Problems*. https://doi.org/10.18280/mmep.120905

41. Wang, H., Zhong, J., Wang, C., (...), Li, X. (2026) SEAL: Self-evolving agentic learning for conversational question answering over knowledge graphs. *Neurocomputing*. https://doi.org/10.1016/j.neucom.2026.133998

42. Wei, J., Wang, X., Schuurmans, D., (...), Zhou, D. (2022) Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *Advances in Neural Information Processing Systems*. https://www.scopus.com/pages/publications/85163157409

43. King, J.C. (1976) Symbolic Execution and Program Testing. *Communications of the ACM*. https://doi.org/10.1145/360248.360252
44. Schneider, F.B. (2000) Enforceable Security Policies. *ACM Transactions on Information and System Security*. https://doi.org/10.1145/353323.353382
45. Shinn, N., Cassano, F., Gopinath, A., (...), Yao, S. (2023) Reflexion: Language Agents with Verbal Reinforcement Learning. *Advances in Neural Information Processing Systems*. https://www.scopus.com/pages/publications/85185875693
46. Lochau, M., Lity, S., Lachmann, R., (...), Goltz, U. (2014) Delta-oriented model-based integration testing of large-scale systems. *Journal of Systems and Software*. https://doi.org/10.1016/j.jss.2013.11.1096
47. Goel, S., Lymperopoulos, P., Thielstrom, R., (...), Scheutz, M. (2024) A neurosymbolic cognitive architecture framework for handling novelties in open worlds. *Artificial Intelligence*. https://doi.org/10.1016/j.artint.2024.104111
48. Liu, X., Yu, H., Zhang, H., (...), Tang, J. (2024) AGENTBENCH: EVALUATING LLMS AS AGENTS. *12th International Conference on Learning Representations, ICLR 2024*. https://www.scopus.com/pages/publications/85198955419
49. Abawajy, J. (2013) SQLIA detection and prevention approach for RFID systems. *Journal of Systems and Software*. https://doi.org/10.1016/j.jss.2012.11.022
50. Zhang, Y., Chen, Z., Guo, Q., Wan, J. (2026) MAMMD: A multi-agent collaboration framework for multimodal metaphor detection. *Neurocomputing*. https://doi.org/10.1016/j.neucom.2026.133381
51. https://www.sciencedirect.com/science/article/pii/S1939165X25000116
52. Cemri, M., Pan, M.Z., Yang, S., (...), Stoica, I. (2025) Why Do Multi-Agent LLM Systems Fail?. *Advances in Neural Information Processing Systems*. https://www.scopus.com/pages/publications/105048535072
53. Zhao, A., Huang, D., Xu, Q., (...), Huang, G. (2024) ExpeL: LLM Agents Are Experiential Learners. *Proceedings of the AAAI Conference on Artificial Intelligence*. https://doi.org/10.1609/aaai.v38i17.29936
54. Chen, M., Li, Y., Yang, Y., (...), He, X. (2024) AutoManual: Constructing Instruction Manuals by LLM Agents via Interactive Environmental Learning. *Advances in Neural Information Processing Systems*. https://www.scopus.com/pages/publications/105000533979
55. Nanda, R., Maddila, C., Jha, S., (...), Chandra, S. (2026) Wink: Recovering from Misbehaviors in Coding Agents. *AIware 2026 - Proceedings of the 3rd ACM International Conference on AI-Powered Software, Co-located with FSE 2026*. https://doi.org/10.1145/3805760.3814911
56. Barrak, A. (2025) Traceability and Accountability in Role-Specialized Multi-Agent LLM Pipelines. *Proceedings - 2025 40th IEEE/ACM International Conference on Automated Software Engineering Workshops, ASEW 2025*. https://doi.org/10.1109/ASEW67777.2025.00064

57. Yin, Z., Feng, Y., Wang, H., An, W. (2027) An agentic framework for reference integrity assurance via multi-source verification and topology-constrained citation consistency. *Information Processing and Management*. https://doi.org/10.1016/j.ipm.2026.105075

58. Wu, H., Wang, Z., Rao, Z., (...), Xiong, H. (2026) U-EHR: A Self-Evolving EHR Agent with Step-Level Credit Assignment and UCB-Guided Memory Retrieval. *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. https://doi.org/10.1145/3770855.3819038

59. Sarukkai, V., Xie, Z., Fatahalian, K. (2025) Self-Generated In-Context Examples Improve LLM Agents for Sequential Decision-Making Tasks. *Advances in Neural Information Processing Systems*. https://www.scopus.com/pages/publications/105048597696

60. Zhang, Liyu, Huang, Dou, Yu, Weichao, (...), Zhang, Haoran (2026) Pipeline-agent: A comprehensive decision support system for natural gas pipelines via verifiable computational reasoning. *Journal of Pipeline Science and Engineering*. https://doi.org/10.1016/j.jpse.2026.100473

61. https://www.sciencedirect.com/science/article/pii/S1600576726000610

62. Zhang, L., Yuan, J., He, Y., (...), Yu, Z. (2026) Diversity-driven reasoning: Mitigating logical errors in LLMs through social-attribute guided multi-agent collaboration. *Engineering Applications of Artificial Intelligence*. https://doi.org/10.1016/j.engappai.2025.113126

63. Zhang, Z., Bu, W., Pan, K., (...), Siliang, S. (2026) Evolving Generalist Virtual Agents with Generative and Associative Memory. *Proceedings of the AAAI Conference on Artificial Intelligence*. https://doi.org/10.1609/aaai.v40i15.38300

64. Ou, T., Guo, W., Gandhi, A., (...), Yue, X. (2025) AgentDiagnose: An Open Toolkit for Diagnosing LLM Agent Trajectories. *EMNLP 2025 - 2025 Conference on Empirical Methods in Natural Language Processing, Proceedings of the System Demonstrations*. https://doi.org/10.18653/v1/2025.emnlp-demos.15

65. Xie, Zhibin, Wang, Hongyu, Dai, Lexuan, (...), Qian, Jingzhe (2026) Ethical issues in multi-agent AI systems for healthcare: a narrative review. *Frontiers in Public Health*. https://doi.org/10.3389/fpubh.2026.1792627

66. Ha, H.-Y. (2026) When rewards buffer service failures: The role of residual value in loyalty programs. *Journal of Hospitality and Tourism Management*. https://doi.org/10.1016/j.jhtm.2026.101530

67. Deriu, Jan, Rodrigo, Alvaro, Otegi, Arantxa, (...), Cieliebak, Mark (2020) Survey on evaluation methods for dialogue systems. *Artificial Intelligence Review*. https://doi.org/10.1007/s10462-020-09866-x

68. DE ANGELIS, E., Fioravanti, F., Pettorossi, A., Proietti, M. (2022) Contract Strengthening through Constrained Horn Clause Verification. *Electronic Proceedings in Theoretical Computer Science, EPTCS*. https://doi.org/10.4204/EPTCS.373.3

69. Fedchin, A., Bai, A.Y., Foster, J.S. (2025) Metamorph: Synthesizing Large Objects from Dafny Specifications. *Proceedings of the ACM on Programming Languages*. https://doi.org/10.1145/3720448

70. Molina, F., Ponzio, P., Aguirre, N., Frias, M.F. (2023) EvoSpex: A Search-Based Tool for Postcondition Inference. *ISSTA 2023 - Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. https://doi.org/10.1145/3597926.3604928

71. Trong, D.N., Nguyen Huu, D.N., Tran, L.N.T., (...), Hung, P.N. (2026) ConAgent: Agent-based concolic testing to generate test data for Qt projects. *Information and Software Technology*. https://doi.org/10.1016/j.infsof.2026.108301

72. Pacella, M., Papadia, G., Giliberti, V. (2026) Governed Agentic Process Automation: A Floor-Safety Guarantee for Compliance-Critical LLM Routing. *Algorithms*. https://doi.org/10.3390/a19080627

73. Bai, Y., Xing, G., Zheng, H., (...), Ren, K. (2026) A systematic analysis of LLM backdoor-integrated cyberattacks: Feasibility, impact factors, and realistic case studies in agent systems. *Applied Soft Computing*. https://doi.org/10.1016/j.asoc.2026.115912

74. Madugundi, C., Muppuri, S., Mitikiri, M. (2026) DAP-L: A Database-Agnostic, Identity-Preserving Streaming Protocol for LLM Agents. *2026 Silicon Valley Cybersecurity Conference, SVCC 2026*. https://doi.org/10.1109/SVCC69905.2026.11642241

75. Zhang, Haiqing, Shang, Yongqiang (2025) Analyzing road traffic crashes through multidisciplinary video data approaches. *Frontiers in Public Health*. https://doi.org/10.3389/fpubh.2025.1614017

76. Graul, C., Żarski, W., Mikołajewski, D., Rojek, I. (2026) AI Agents in Industry 4.0: AAS–OPC UA–LLM Architecture as the Foundation of Intelligent Manufacturing Systems in the Context of Industrial Enterprise Implementation. *Applied Sciences (Switzerland)*. https://doi.org/10.3390/app16115428

77. Stylianou, I., Bountakas, P., Zarras, A., (...), Xenakis, C. (2026) SoK: A taxonomy of LLM threats. *Computer Science Review*. https://doi.org/10.1016/j.cosrev.2026.101013

78. Debenedetti, E., Zhang, J., Balunović, M., (...), Tramèr, F. (2024) AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. *Advances in Neural Information Processing Systems*. https://www.scopus.com/pages/publications/105000481564

79. Briand, L.C., Labiche, Y., He, S. (2009) Automating regression test selection based on UML designs. *Information and Software Technology*. https://doi.org/10.1016/j.infsof.2008.09.010

80. Su, Z., Luo, M., Tang, Y., (...), Ding, X. (2026) RL-Based Test Case Prioritization with LSTM-Attention Reward Function. *Lecture Notes of the Institute for Computer Sciences, Social-Informatics and Telecommunications Engineering, LNICST*. https://doi.org/10.1007/978-3-032-21168-2_16

81. Kuhn, R., Kacker, R. (2011) Practical combinatorial (t-way) methods for detecting complex faults in regression testing. *IEEE International Conference on Software Maintenance, ICSM*. https://doi.org/10.1109/ICSM.2011.6080840

82. Trad, F., Chehab, A. (2026) Toward accurate and cost-effective LLM agents via information flow optimization: Insights from a phishing detection case study. *Information Processing and Management*. https://doi.org/10.1016/j.ipm.2026.104834

83. Gao, Y., Hu, F., Chai, C., (...), Li, H. (2026) Multi-agent framework for schema-guided reasoning and tool-augmented interaction with IFC models. *Automation in Construction*. https://doi.org/10.1016/j.autcon.2026.106888

84. Ghosh, S., Tewari, A. (2026) LLM-based AI agents for automated extraction of material properties and structural features. *Computational Materials Science*. https://doi.org/10.1016/j.commatsci.2026.114521

85. Wang, J., Fu, G., Savić, D. (2026) Leveraging large language models for automating water distribution network optimization. *Water Research*. https://doi.org/10.1016/j.watres.2025.124536

86. Trad, F., Chehab, A. (2026) Toward accurate and cost-effective LLM agents via information flow optimization: Insights from a phishing detection case study. *Information Processing and Management*. https://doi.org/10.1016/j.ipm.2026.104834

87. Zhou, S., Han, W., Xiong, L. (2026) Road-adaptive reinforcement learning for safe active suspension control. *Applied Soft Computing*. https://doi.org/10.1016/j.asoc.2026.116023

88. Li, M., Li, J., (...), Zhang, S. (2026) NIDW: A post-hoc cascade multi-channel text watermarking framework with Newton interpolation. *Knowledge-Based Systems*. https://doi.org/10.1016/j.knosys.2026.117089

89. Pereira, M., Gaudet, M., Amaral, J., Araújo, G. (2016) Study of hardware transactional memory characteristics and serialization policies on Haswell. *Parallel Computing*. https://doi.org/10.1016/j.parco.2015.12.002